

Algorithms for Data Science

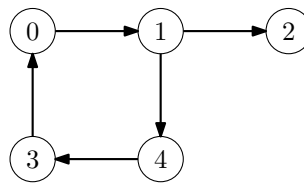
Exercise 6

(with Solutions)

See the cheatsheet for valuable remarks!

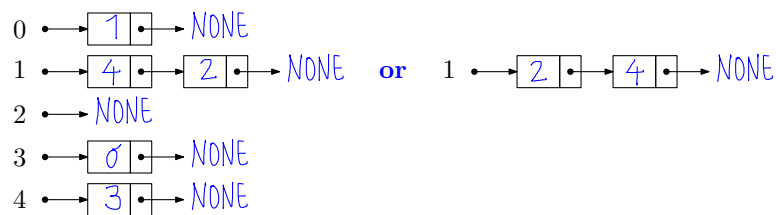
Task 6.1: (Simple Tasks) Graphs, Stack and BFS

(a) Given the following directed graph, write the corresponding adjacency matrix and adjacency lists.



Solution:

Adjacency lists:



Adjacency matrix:

	0	1	2	3	4
0	0	1	0	0	0
1	0	0	1	0	1
2	0	0	0	0	0
3	1	0	0	0	0
4	0	0	0	1	0

(b) Write the outputs of the following sequence of operations when the underlying data structure is an empty Stack.

Add(0), Add(4), FindNewest(), Add(3), ExtractNewest(), Add(2), ExtractNewest(), Add(1), ExtractNewest(), ExtractNewest(), FindNewest(), Add(2), ExtractNewest(), ExtractNewest()

Solution: Only the operations FindNewest() and ExtractNewest() make outputs. We obtain:

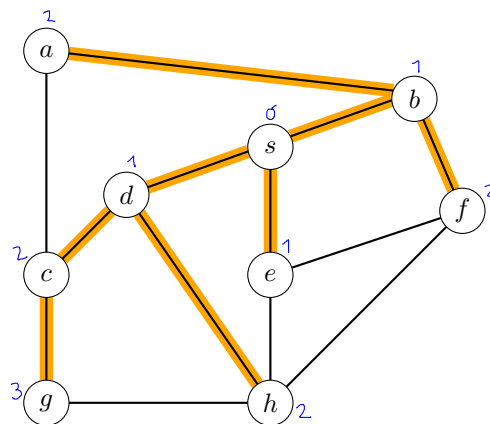
4, 3, 2, 1, 4, 0, 2, 0

- (c) Breadth-first Search (BFS) is run on the following graph with starting node s . Order the vertices by their order of being extracted from the queue and compute their distances and parents. (If a vertex has more neighbors, we consider them in alphabetical order.)

```

dist[s] = 0
parent[s] = NONE
q = new queue
q.add(s)
while q not empty do
    v = q.extractOldest()
    for every edge (v,w) do
        if dist[w] not computed yet
            then
                dist[w] = dist[v] + 1
                parent[w] = v
                q.add(w)

```



Solution:

	s	a	b	c	d	e	f	g	h
distance	0	2	1	2	1	1	2	3	2
parent	NONE	b	s	d	s	s	b	c	d
removal order	1	5	2	7	3	4	6	9	8

Task 6.2: Drawing Graphs

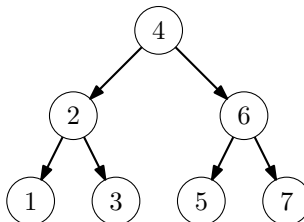
- (a) Draw a directed graph without edge crossings corresponding to the following adjacency lists.

```

1 → NONE
2 → [1] → [3] → NONE
3 → NONE
4 → [2] → [6] → NONE
5 → NONE
6 → [5] → [7] → NONE
7 → NONE

```

Solution:

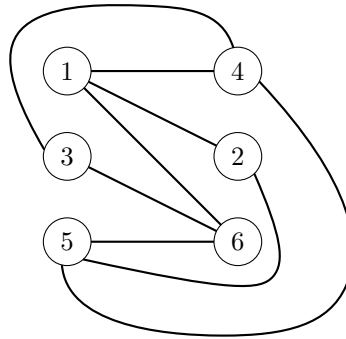


Coincidentally or not, the graph looks like a complete binary search tree.

(b) Draw a (undirected) graph without edge crossings corresponding to the following adjacency matrix.

	1	2	3	4	5	6
1	0	1	0	1	0	1
2	1	0	0	0	1	0
3	0	0	0	1	0	1
4	1	0	1	0	1	0
5	0	1	0	1	0	1
6	1	0	1	0	1	0

Solution:



A side note: The graph is a *bipartite* graph and it nearly corresponds to the *complete* bipartite graph $K_{3,3}$. In a bipartite graph, the vertices belong to two groups and there are only edges between the two groups. In our case, the vertices with odd and even numbers form two such groups. A complete bipartite graph contains all possible edges between the two groups. In our case, we miss the edge $(2, 3)$ to be complete. And this is no coincidence as the complete bipartite graph with three vertices in each group, called $K_{3,3}$, cannot be drawn without crossings. You do not believe? Then try it! Or rather try to prove that it is not possible.

The two smallest graphs that cannot be drawn without crossings are $K_{3,3}$ and the graph K_5 which is defined as the graph that has five vertices and contains every possible edge. A famous result in graph theory states that a graph is planar (i.e., it can be drawn without edge crossings) if only if it does neither contain $K_{3,3}$ nor K_5 as a so-called *minor* (a subgraph obtained via edge deletions and so-called edge contractions). Interestingly, one can decide in linear time whether a given graph is planar and even compute a planar (i.e., crossing-free) drawing in the same time!

Task 6.3: *Waiting in the Queue!*

Write the outputs of the following sequence of operations when the underlying data structure is an empty Queue.

Add(0), Add(4), FindOldest(), Add(3), ExtractOldest(), Add(2), ExtractOldest(), Add(1), ExtractOldest(), ExtractOldest(), FindOldest(), Add(2), ExtractOldest(), ExtractOldest()

Solution: The output writes the elements in the order of how they have been added, thus:

0, 0, 4, 3, 2, 1, 1, 2

Task 6.4: Dijkstra

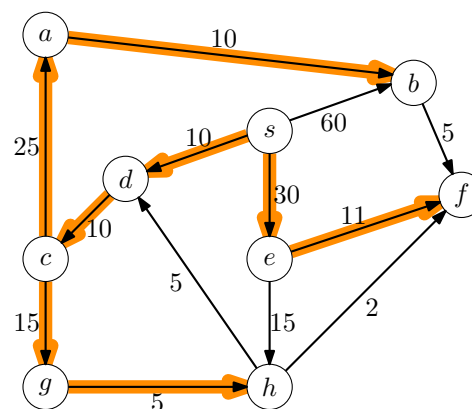
Dijkstra's algorithm (see below) is run on the following graph with starting node s . Compute the distances and parents of all nodes. Also mark all edges that belong to cheapest paths from s .

Hint: While solving, write the current distances next to the vertices and mark those vertices that have already been removed from the queue.

```

dist[0..n - 1] filled with INF
parent[0..n - 1] filled with NONE
dist[s] = 0
q=new priority queue
q.add(s,0)
while q not empty do
    v = q.extractMax()
    if v removed for the first time
        then
            for every edge (v,w) do
                if dist[w] > dist[v] + cv,w
                    then
                        dist[w] = dist[v] + cv,w
                        parent[w] = v
                        q.add(w, -dist[w])

```



Solution:

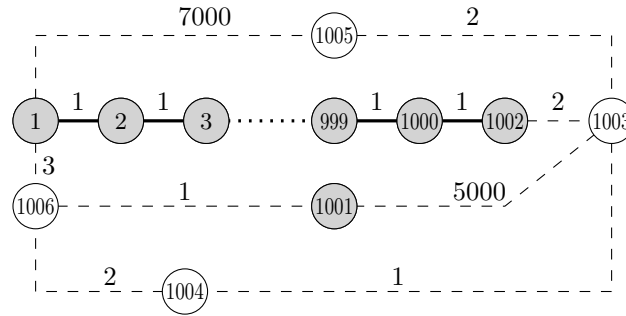
	s	a	b	c	d	e	f	g	h
distance	0	45	55	20	10	30	41	35	40
parent	NONE	c	a	d	s	s	e	c	g

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 6.5: Incomplete Floyd-Warshall

Floyd-Warshall's algorithm (see below) is executed on the following huge undirected graph where the vertices are numbered from 1 to $n=1006$ and the edge weights are given as below (the edges on the path from 3 to 999 have weight 1).



Unfortunately, the algorithm as written below has an **error** because it stops the main for loop (line 9) already after 1002 iterations.

```

1  $\delta[1..1006][1..1006]$  = new 2D array filled with  $+\infty$ 
2 for  $i = 1$  to 1006 do
3    $\delta[i][i] = 0$ 
4 foreach  $edge(v, w, c)$  do
5   //  $v$  and  $w$  are the endpoints and  $c$  is the weight of the edge
6    $\delta[v][w] = c$ 
7    $\delta[w][v] = c$ 
7 for  $k = 1$  to 1002 do
8   foreach  $pair(v, w)$  do
9      $\delta[v][w] = \min(\delta[v][w], \delta[v][k] + \delta[k][w])$ 
10 return  $\delta$ 

```

Write the values of the distance table for the following cells as computed by the **erroneous** algorithm above. That is, we still assume that the loop in line 7 iterates **only 1002 times!**

$\delta[1][1002] =$ 1000 $\delta[1][1001] =$ $+\infty$ $\delta[1][1005] =$ 7000
 $\delta[2][4] =$ 2 $\delta[1003][1006] =$ 1005 $\delta[1001][1005] =$ $+\infty$
 $\delta[1003][1004] =$ 1 $\delta[1001][1004] =$ $+\infty$

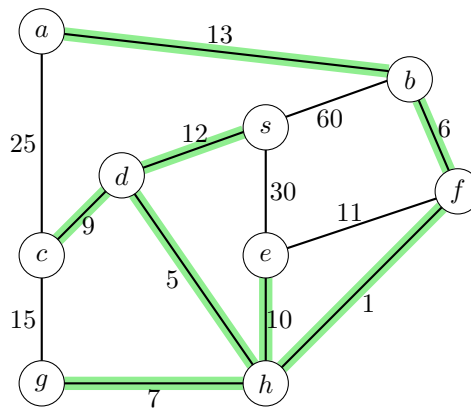
Solution: According to the invariant that we discussed in the lecture, after 1002 iterations ($k = 1002$), the Floyd-Warshall algorithm has computed for every pair of vertices the cost of a cheapest path that uses only vertices 1 to 1002 (shaded gray in the figure). In other words, for each pair of

vertices that we are asked for, we look for a cheapest path that is either a direct edge or that uses internally only gray vertices. If there is no such path, we output infinity!

Task 6.6: *Minimum Spanning Tree*

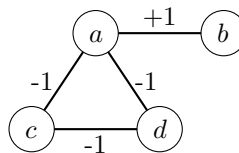
- (a) Draw a minimum spanning tree in the following graph.

Solution: Take the cheapest edge that does not produce a cycle, and repeat.



- (b) A minimal spanning subgraph is always a tree if the edge weights are positive. Draw a weighted graph that has positive and negative edge weights and whose minimal spanning subgraph is not a tree.

Solution: For instance, in the following graph, a minimal spanning subgraph has to take all the edges and thus contains a cycle.



Task 6.7: *Salt and Pepper*

For cooking one needs only salt and pepper. Among a group of n people, give every person either a salt shaker or a pepper shaker such that when any two friends meet, they can cook together. If such a distribution is not possible, say so. In total there are k pairs of friends. (It is possible that one person has several friends in the group.)

Model the problem as a graph problem and use (or modify) an appropriate graph algorithm from the lecture to solve it. Your solution should work in time $O(n + k)$.

- (a) First, assume that your graph is connected!
- (b) Next, give an algorithm that also works if the graph is not connected! (*Advanced task:* is your running time really $O(n + k)$?)

Solution: Solution sketch (for both cases: that the graph is connected and not necessarily connected):

- vertices = people
- edges = pairs of friends
- Color problem: Say whether it is possible to color the vertices in white and black such that no two neighboring vertices have the same color. (If it is possible, return such a coloring.)
- Observation 1: If there is a solution, then we can swap black and white and it is still a solution.
- Observation 2: If a vertex has one color, then all its neighbors must have the other color.
- Observation 3 (a consequence of Observation 2): If a vertex s is, say, black, then all vertices with odd distance to s must be colored white and all vertices with even distance must be colored black (provided that a solution exists).
- Algorithm 1: For each vertex, if it is not colored yet, run BFS from it and color all visited vertices according to Observation 3 (choosing any color for the start vertex; see Observation 1). At the end, go through every edge and check whether its endpoints have different colors. Total time as required.
- Note: If the graph is connected, the foreach loop of Algorithm 1 will run only once (we take an arbitrary vertex as our start vertex and then reach and color all the other vertices).
- Modification: While running BFS, we can already check whether the coloring is valid.

Running time analysis:

- BFS takes $O(n + k)$ time. (Recall that k is now the number of edges.)
- Checking at the end whether the endpoints of each edge have different colors takes $O(k)$ time.
- If the graph is connected (we are allowed to assume it), we run BFS only once and the total time is thus $O(n + k)$.
- If the graph is not connected, we run BFS several times. (*This is the solution of an advanced task and I don't expect you to come up with such an analysis!*) The fact that we run BFS several times might lead to higher total running time than $O(n + k)$. For instance, if the graph contains no edges at all, then we run BFS n times. This would lead to a running time of at least $\Omega(n^2)$ as, each time, BFS has to initialize an empty distance table of length n which already takes $\Theta(n)$ time. However, we can modify BFS such that, at each subsequent call, it uses the same distance table from the last call (without changing the values). If BFS does not have to initialize the distance table, then its running time drops to $O(n_s + k_s)$ where n_s and k_s are the number of vertices and edges, respectively, considered by BFS which are exactly the vertices and edges reachable from the starting vertex s . Hence, if S is the set of all vertices from which we run BFS in our algorithm (their number equals the number of connected components in the graph and, in particular, we have $\sum_s n_s = n$ and $\sum_s k_s = k$), then the total running time of our algorithm is (which consists of the time $O(n)$ to initialize the distance table, then the time to run BFS from every vertex from S , as well as to check all edges at the end):

$$O(n) + \sum_s O(n_s + k_s) + O(k) = O(n) + O(n + k) + O(k) = O(n + k)$$

Task 6.8: *Is it true ...? (Graphs)*

- (a) Is it true that any connected graph with $n - 1$ edges has no cycles? Prove the claim or draw a counter example.

Solution: It is true.

One of many possible proofs:

Suppose for contradiction that there is a cycle. As long as there is a cycle, remove an edge from the cycle. Note that the graph still is connected afterwards but now it has less than $n - 1$ edges. In other words, there are at least two more vertices than edges.

Next, as long as there is a vertex with exactly one edge, delete the vertex and the edge. By this operation, we decrease the number of vertices by one and the number of edges by one, and the remaining graph is still connected and still has at least two more vertices than edges. Note that the remaining graph has at least one vertex.

If the process stops because all remaining vertices have at least two edges, then we actually have cycle. But recall that we have already destroyed all cycles. So this situation can't happen.

Thus, the process has to stop because there are no edges left. Consider the moment when we remove the last edge in this process. As shown above, there are at least two more vertices than the number of edges. Hence, after removing the last edge with exactly one of its endpoint, we are left with two vertices and no edge. Hence, the remaining graph is disconnected which contradicts our observation that it has to be connected after each step of the process. This means that our initial assumption that the initial graph had a cycle is wrong. Hence, there are no cycles in the initial graph.

Note that connected (undirected) graphs with $n - 1$ edges are exactly the graphs that we call trees.

- (b) Also in this task, we consider only graphs without parallel edges. What is the maximum number of edges ...
- ... in a directed graph when we allow self-loop edges?
 - ... in a directed graph when we do not allow self-loop edges?
 - ... in an (undirected) graph when we do not allow self-loop edges?

Solution: The answers are: n^2 , $n(n - 1)$, $n(n - 1)/2$.

Explanations: (i) If we allow self loop edges, then every vertex has a directed edge to every other vertex (including itself). Hence, there are $n \cdot n = n^2$ edges.

(ii) As above, but now every vertex has only a directed edge to every other vertex (not including itself). Hence, there are $n \cdot (n - 1)$ edges.

(iii) As above, but we have to divide it by 2 as above we counted every edge twice. In other words: For every undirected edge, we counted two directed edges above (one in one direction and the other one in the other one).

Task 6.9: *Floyd-Warshall vs Dijkstra*

In the lecture, we have seen that the problem of computing the cheapest path from every vertex to every other can be solved via the Floyd-Warshall algorithm in time $\Theta(n^3 + m)$ or by running Dijkstra's algorithm from every vertex in total time $\Theta(mn \log m + n^2)$ (on the slides we further simplified these running times assuming that there are no parallel edges).

What are the running times of the two approaches if ...

- (a) ... m is extremely small, that is, $m = \Theta(1)$? Give the running times in dependence of only n .

Solution: If $m = O(1)$, then Floyd-Warshall runs in $\Theta(n^3)$ and Dijkstra in $\Theta(n^2)$.

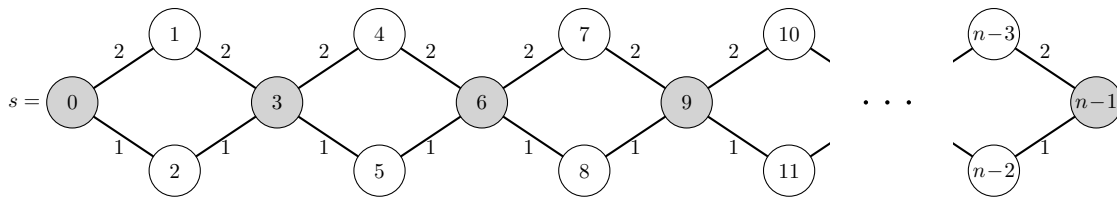
- (b) ... m is extremely large, that is, $m = \Theta(n^4)$ (thus, we have parallel edges)? Give the running times in dependence of only m .

Solution: If $m = \Omega(n^4)$, then Floyd-Warshall runs in $\Theta(m)$ and Dijkstra in $\Theta(m^{1+1/4} \log m)$.

Task 6.10: Dijkstra with a Normal Queue

In this task, you learn why it is very important to use a priority queue in Dijkstra's algorithm.

Consider the following graph. Argue that, in the worst case, Dijkstra's algorithm will run immensely much slower on that graph if we use a normal queue (as in the algorithm below) instead of a priority queue!



```

dist[0..n-1] filled with INF
dist[s] = 0
q = new queue
q.add(s)
while q not empty do
    v = q.removeOldest()
    if v removed for the first time then
        for every edge (v, w) do
            if dist[w] > dist[v] + cv,w then
                dist[w] = dist[v] + cv,w
                q.add(w)

```

Solution: You will notice that in the worst case, if a vertex has more than one neighbor, we consider them from top to bottom (equivalently, in increasing order of their numbers). Thus, first the neighbor with the more costly edge, then the neighbor with the cheaper edge. If you try to run the algorithm by hand, you will notice that the vertices are added to the queue in this order:

0, 1, 2, 3, 3, 4, 5, 4, 5, 6, 6, 6, 6, ...

You see that the same vertices are added over and over again. If you focus on the gray vertices (in bold above), which are exactly the vertices whose numbers are divisible by 3, you will see that each next gray vertex is added twice as often as the preceding gray vertex. In other words, the gray vertex with number $3k$ is added 2^k times to our queue. (You can also prove this fact with induction—see the cheatsheet to the first exercise sheet.) Consequently, the last gray vertex with the number $n-1$ is added $2^{(n-1)/3} = \Theta(c^n)$ times, where c is the constant $\sqrt[3]{2}$. Hence, the last vertex is added exponentially often.

Thus, the total running time is at least exponential, which is extremely much slower than Dijkstra's almost linear running time of $O(n + m \log n)$.

Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 6.11: Output the Cheapest Paths Using Floyd-Warshall

- (a) Extend the algorithm of Floyd-Warshall such that it outputs the actual cheapest paths in additional time $O(n)$ for each pair.

Note: Print the vertices as they appear on the path, one by one.

Solution: Modify Floyd-Warshall to also compute a 2D table $next$ such that $next[u][v]$ is the successor of u on the cheapest path from u to v . (Initially, set $next[u][v] = v$ for every pair u, v .) Whenever you set $\delta[u][v] = \delta[u][k] + \delta[k][v]$ (which happens only if $u \neq k$), update $next[u][v] = next[u][k]$.

Once $next$ is computed, you can recursively reconstruct a cheapest u - v -path as follows:

```
k = u
while k ≠ v do
    Print(k)
    k = next[k][v]
Print(k)
```

The time is obviously $O(n)$ for reconstruction. The time to compute $next$ does not increase the $O(n^3)$ time of Floyd-Warshall.

- (b) Now suppose that you are only given the δ table as computed by Floyd Warshall and that you don't have access to the graph. Give an $O(n^2)$ -time algorithm that, given a pair of vertices u and v , reconstructs a cheapest path from u and v . Assume that the δ table already lies in the memory and that it takes $O(1)$ time to read a single entry of the table.

Note: Print the edges as they appear on the path, one by one.

Solution: Backtrack the Floyd-Warshall algorithm by reconstructing which decisions had been made. You can do it, for example, recursively as follows.

```
ReconstructPath(u, v)
    return AlgoRec(u, v, n)

AlgoRec(u, v, k)
    if k == 0 then
        if u ≠ v then
            Print((u, v))
    else
        if δ[u][v] == δ[u][k] + δ[k][v] then
            AlgoRec(u, k, k - 1)
            AlgoRec(k, v, k - 1)
        else
            AlgoRec(u, v, k - 1)
```

Or you can do it iteratively using a stack; which, by the way is a nice example of how stacks are useful to transform a recursive algorithm into an iterative one. In fact, a recursive algorithm

has always a so-called recursion stack so that the computer remembers how to go back and what to do next when coming back from a recursion.

```

s = new empty stack
s.Add((u, v, n))
while s not empty do
    (u, v, k) = s.ExtractNewest()
    if k == 0 and u ≠ v then
        Print((u, v))
    else if k > 0 then
        if δ[u][v] == δ[u][k] + δ[k][v] then
            s.Add(k, v, k - 1)
            s.Add(u, k, k - 1)
        else
            s.Add(u, v, k - 1)

```

To analyze the time, first observe that the condition $\delta[u][v] == \delta[u][k] + \delta[k][v]$ is satisfied at most once for every k , hence, it happens at most $O(n)$ times. Thus, at most $O(n)$ times, we start to consider two new vertex pairs. Each vertex pair is considered not more than n times as k decreases by 1 each time. Hence, we have at most $O(n^2)$ triples that we consider (and each triple is considered at most once). This discussion implies the running time.